

On Bit-level Reverse Engineering of Vehicular CAN Bus

Yunlang Cai^{1,2,#}, Hanxue Shi^{1,2,#}, Xiaohang Wang^{1,2,*}, Haoting Shen^{1,2,*}, Li Lu^{1,2}, Kui Ren^{1,2}

¹State Key Laboratory of Blockchain and Data Security, Zhejiang University, China

²Hangzhou High-Tech Zone (Binjiang), Institute of Blockchain and Data Security

17620486393@163.com, sarahshi99@163.com, xiaohangwang@zju.edu.cn, htshen@zju.edu.cn, li.lu@zju.edu.cn, kuiren@zju.edu.cn

Abstract—The Controller Area Network (CAN) bus is a cornerstone of modern vehicles, orchestrating functions from engine control to auxiliary systems. However, its lack of inherent security measures makes it vulnerable to cyberattacks. Accurately mapping CAN signals with car-control actions is critical for detecting security breaches, as it allows pinpointing potential vulnerabilities exploited to compromise vehicular functions. Despite this, existing CAN reverse engineering methods struggle to achieve bit-level resolution due to the huge search space of unique IDs and payload combinations. To address this challenge, we propose a systematic framework for reverse engineering CAN bus messages, achieving precise mapping of control bits in CAN frames to car-control actions. The framework was validated on Tesla Model 3, Leapmotor C10 and C11, demonstrating its versatility across different vehicle platforms. In particular, it successfully identified 43 car-control actions on the Tesla Model 3, showcasing its extensive coverage. Furthermore, its low resource consumption enables seamless integration into compact platforms like the Raspberry Pi, supporting practical deployment in real-world automotive systems.

Keywords—CAN Bus Reverse Engineering, Signal Boundary Identification, Fuzz Testing

I. INTRODUCTION

The Controller Area Network (CAN) bus is a cornerstone of modern vehicle communication, orchestrating functions from critical engine control to auxiliary systems like lighting and climate management. Its standardized design ensures reliability and efficiency, but also introduces a significant vulnerability: the lack of inherent security measures. As vehicles increasingly rely on software-driven control and connectivity features, this vulnerability leaves them exposed to a variety of cyber threats. Attackers could exploit CAN's openness to inject malicious messages, potentially compromising vehicle safety and user privacy.

Each automotive manufacturer (OEM) uses proprietary DBC files to define the meaning of CAN signals, obscuring the internal logic of its systems and masking unique security flaws. This fragmentation not only complicates cross-manufacturer diagnostics but also hinders the identification and mitigation of potential vulnerabilities. Reverse engineering of CAN bus messages is essential for overcoming these obstacles. By decoding the proprietary definitions in DBC files, reverse engineering enables a detailed interpretation of CAN signals, uncovering hidden security flaws and supporting the development of effective cybersecurity measures.

However, achieving fine-grained decoding of CAN signals faces significant challenges. As illustrated in Figure 1, data frames are continuously transmitted on the CAN bus,

each containing an ID field and a payload field. The ID field determines the frame's priority, while the 64-bit payload field indicates the specific information carried by the frame. Within the payload, manufacturers predefine various fields to represent different command content or status information, as seen in the example fields in the figure. Given that a typical vehicle has hundreds of unique IDs and each payload can have up to 2^{64} possible combinations, the search space for identifying precise control bits of car-control actions is huge. This huge search space creates a major barrier to mapping individual bits to specific car-control actions, which is crucial for identifying potential vulnerabilities.

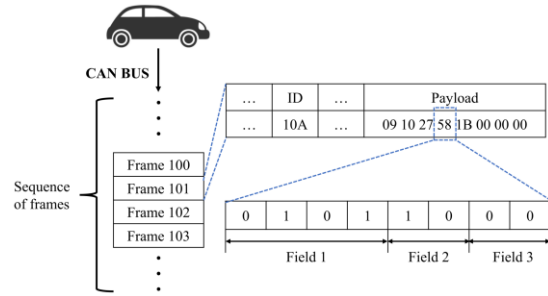


Fig. 1. CAN Bus Data Frame Layout and Search Space Complexity

Although there were many CAN bus signal reverse engineering methods proposed, they cannot address this huge search space problem in CAN signal reverse engineering. Signal boundary identification methods [7–12] delineate broader data field boundaries within CAN messages but do not establish relationships to specific car-control actions or achieve bit-level resolution. Machine learning approaches [13], [17], while helpful in identifying CAN message IDs related to actions, stop at the ID level and lack the capability to decode payload details. OBD-II protocol-based methods [14–16] rely on diagnostic data, which restricts their ability to handle non-diagnostic signals. Together, these limitations underscore the need for a high resolution (bit level) interpretation of the CAN bus, which is vital to pinpoint the car-control actions.

To address these limitations, we present a comprehensive solution designed to efficiently decode CAN signals with high coverage and precision. Our systematic framework for reverse engineering CAN bus messages effectively manages the challenge of huge search spaces while achieving bit-level resolution. By analyzing signal boundaries across different vehicle states, such as stationary or during specific car-control actions (e.g., activating windshield wipers), the framework filters out irrelevant IDs. Targeted fuzz testing further explores payload variations to trigger car-control actions, refining the resolution to the frame level. Finally, control bit analysis isolates the exact bits responsible for specific actions, completing the mapping with bit-level resolution. This streamlined, step-by-step approach not only overcomes technical obstacles but also ensures lightweight and adaptable deployment on compact platforms like the Raspberry Pi. Validated across multiple vehicle models, it proves highly

* Xiaohang Wang and Haoting Shen are the corresponding authors.

The two authors contribute equally to this work and should be regarded as co-first authors.

This work was supported in part by the key R&D programme of Zhejiang Province No. 2024C01012, in part by the National Key Research and Development Program of China No. 2023YFB4404404, in part by the National Natural Science Foundation of China under Grants 92373205 and 62374146, in part by Ant Group Research Fund, in part by the Key Technologies R&D Program of Jiangsu (Prospective and Key Technologies for Industry) under Grant BE2023005-2, and in part by CIE-Smartchip research fund No. 2023-004.

practical and robust, making it a strong candidate for real-world CAN signal reverse engineering tasks.

The contributions of this paper are threefold:

- We introduce a systematic framework combining state-based signal boundary identification, targeted fuzz testing, and control bit analysis. This approach effectively narrows down the search space for relevant CAN signals, achieving precise bit-level resolution in mapping control bits to car-control actions. This not only overcomes the inherent complexity of CAN signal reverse engineering but also significantly improves accuracy compared to existing methods.
- The proposed framework is highly efficient, with low CPU and memory requirements, enabling seamless integration into compact devices like Raspberry Pi systems. We prototyped a wireless automatic reverse engineering tool (using Raspberry Pi). This design supports on-site and portable automotive testing.
- Our method has been validated on three distinct vehicle models, including Tesla Model 3 and Leapmotor C10/C11. It successfully identified 43 car-control actions in a single model Tesla Model 3, achieving extensive coverage with high adaptability. These findings highlight the framework's practicality and versatility across diverse automotive platforms.

II. RELATED WORK

A. Signal Boundary Identification

Signal boundary identification methods [7–12] focus on delineating data field boundaries within CAN messages but vary in their resolution and scope. The READ framework [7] introduced the first automated approach using bit-flip rate analysis, establishing the foundation for subsequent advancements. Building on READ, LibreCAN [8] optimized boundary identification and linked car-control actions with CAN IDs, though it did not achieve bit-level resolution. Choi et al. [9] further refined boundary accuracy with statistical analysis of bit-flip time series but lacked action-specific mapping. Later methods, such as CA-CRE [10] and ACRE [11], employed classification and heuristics to improve format identification but remained limited in granularity. Karamolegkos et al. [12] extended this line of research to protocol-level reverse engineering but similarly fell short of identifying precise control bits.

B. Machine Learning-Based Approaches

Machine learning approaches assist in identifying CAN message IDs related to car-control actions but generally stop at the ID level, lacking the capability to decode payload details. Buscemi et al. [16] applied pattern recognition to identify ID-action relationships but do not decode payload bits. Young et al. [17] used unsupervised learning to classify CAN message IDs, improving functional ID recognition but without detailed payload analysis.

C. OBD-II Diagnostic Protocol-Based Methods

The OBD-II diagnostic protocol, widely used for vehicle diagnostics, provides access to real-time data from the electronic control unit (ECU). In OBD-II-based reverse engineering, Kang et al. [13] introduced an automated tool that correlates OBD-II responses with CAN traffic to identify basic control signals. Building on this, Blaauwendraad and Kieberl [14] applied statistical correlation methods to improve

the accuracy of matching OBD-II data to CAN signals across different vehicles. Song and Kim [15] advanced this approach by mapping specific signal boundaries within CAN data to car-control actions, offering a more detailed view of CAN signal content. However, all these methods are fundamentally constrained by the OBD-II protocol's request-response structure, which limits reverse engineering to predefined Parameter Identifiers (PIDs). Since the ECU only responds to specific, and manufacturer-defined PIDs, the scope of reverse-engineered information is restricted to the data fields allowed by these definitions, excluding many non-diagnostic control signals crucial for comprehensive CAN analysis.

III. THE PROPOSED REVERSE ENGINEERING METHOD

A. Overview

The proposed method uses both state-based signal boundary identification and targeted fuzz testing, systematically narrowing down the search space to locate control bits for specific car-control actions. The workflow begins with collecting CAN messages under stationary and operational states, where the stationary state provides baseline data and the operational state captures vehicle control actions such as door unlocking and window adjustments, as shown in Figure 2. After grouping messages by CAN ID, payload fields are extracted and organized into a matrix for further analysis (in section III. B).

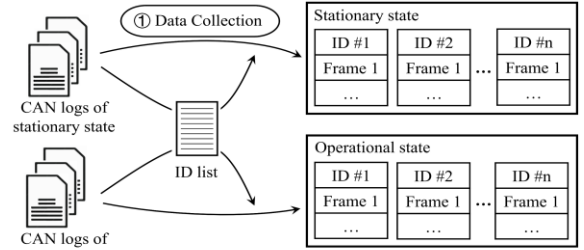


Fig. 2. State-Based CAN Message Collection and Preprocessing Workflow

Signal boundaries are identified using bit-flipping rate (BFR) analysis, which classifies fields into constant, counter, random, and co-flipping types (in section III. C). As illustrated in Figure 3, this classification highlights key areas for subsequent exploration and provides a foundation for related-ID locating. By comparing signal classifications across states, the related-ID locating step identifies IDs exhibiting state-dependent changes, capturing IDs most likely to be relevant for car-control actions (in section III.C.2).

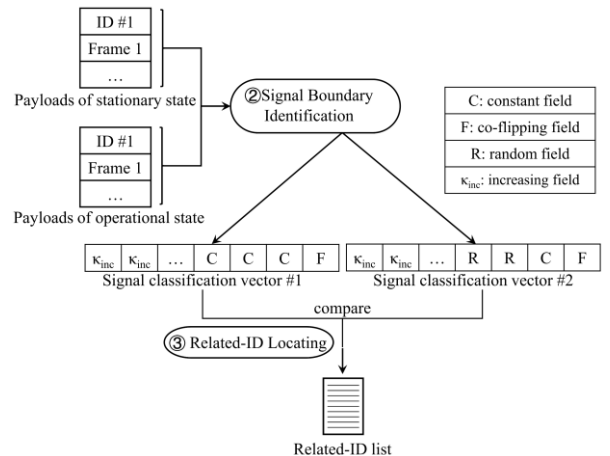


Fig. 3. Signal Boundary Identification and Related-ID Locating Workflow

Fuzzed data is then generated and injected into the CAN bus to provoke specific function controls or abnormal responses (in section III.D). Binary search techniques isolate the responsible frames and bits, enabling both validation and discovery of car-control actions, as shown in Figure 4. Finally, control bit analysis, comprising commonality analysis and selective bit testing, confirms the role of each individual bits in initiating specific actions (in section III.E). This approach achieves bit-level resolution, offering a significant advancement in reverse engineering techniques.

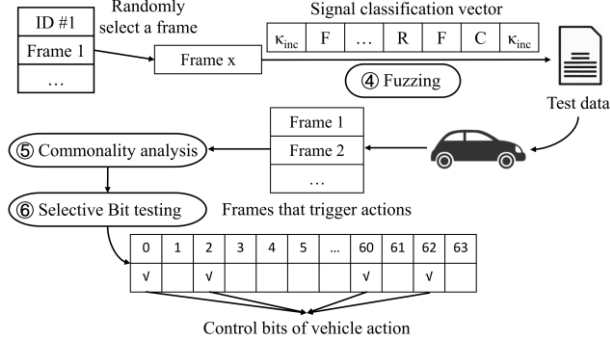


Fig. 4. Fuzz Testing and Control Bit Analysis Workflow

The notations used in this paper are listed in Table I.

TABLE I. NOTATIONS USED IN THIS PAPER

Notations	Description
M_s	CAN data messages collected in the stationary state
M_o	CAN data messages collected in the operational state
Ω_{op}	Set of specific car-control actions in the operational state, such as unlocking, wiper control
$P_{d,s/o}$	A bit matrix representing the payloads of CAN ID d under stationary state (s) or operational state (o). $P_{d,s/o}[i]$ is the i -th column of bits within $P_{d,s/o}$ and $P_{d,s/o}[i][j]$ is the i -th bit of the j -th frame within $P_{d,s/o}$
$bfr_{d,s/o}$	A bit-flipping rate vector for $P_{d,s/o}$ and $bfr_{d,s/o}[i]$ is the bit flipping rate for $P_{d,s/o}[i]$ (The i -th column of bits within $P_{d,s/o}$)
$\tau_{d,s/o}$	A signal classification vector that maps each bit index i in the payload $P_{d,s/o}$ to its types: constant (C), incrementing counter (κ_{inc}), decrementing counter (κ_{dec}), co-flipping (F_k), or random (R). The i -th element $\tau_{d,s/o}[i]$ is the classification of the i -th bit in $P_{d,s/o}$
$\Sigma_{related}$	Set of IDs with signal boundaries that differ between states, possibly associated with specific car-control actions
$P_{d,fuzzed}$	A bit matrix of fuzzed payloads for CAN ID d generated through a mutation-based fuzzer. Each row $P_{d,fuzzed}[i]$ represents a fuzzed payload as a 64-bit vector.
$M_{d,fuzzed}$	Complete fuzzed data messages for CAN ID d , ready for injection
ω_i	A specific car-control action
Ω_{tr}	Set of all ω_i observed during the injection process
Φ_{d,ω_i}	Set of payload vectors for CAN ID d that independently trigger car-control action ω_i
χ_{d,ω_i}	A marked vector produced by commonality analysis on Φ_{d,ω_i} .
$\bar{\chi}_{d,\omega_i}^{(i)}$	A test vector derived from χ_{d,ω_i} by flipping the i -th bit, while fixing all x -marked positions to 0. Used in selective bit testing.
$\mathcal{C}_{d,\omega_i}$	Set of control bits for ω_i , representing the indices of bits in the payload of CAN ID d frames that directly

Notations	Description
ω_i	control ω_i . This set is determined through selective bit testing.

B. Data Collection and Preprocessing

To gather the necessary data for reverse engineering, CAN messages are collected in two distinct states: a stationary state, denoted as M_s , with no active car-control actions, and an operational state, denoted as M_o , with car-control actions which is denoted as Ω_{op} , e.g., unlocking the car, controlling the windows, or activating the windshield wipers.

During the preprocessing phase, the collected CAN messages are grouped by CAN ID. For each CAN ID n , the payload, denoted as $P_{n,s}$ or $P_{n,o}$, is extracted under stationary or operational state, respectively. $P_{n,s/o}$ is a bit matrix, where each row represents the 64-bit payload of a single frame, and each column corresponds to a specific bit position. The matrix has N rows, where N represents the number of frames with ID n in $M_{s/o}$ (M_s or M_o). In $P_{n,s/o}$, the bit index i starts from 0 on the leftmost column, corresponding to the most significant bit (MSB), and increases to 63 on the rightmost column, corresponding to the least significant bit (LSB).

C. Bit-Flipping Analysis Model

This step leverages the bit-flipping rate (BFR) to classify and differentiate signal fields within CAN messages. For each ID n and its payload $P_{n,s/o}$, the bit-flipping rate for $P_{n,s/o}[i]$ (the i -th column of bits within $P_{n,s/o}$) is denoted as $bfr_{n,s/o}[i]$, representing the percentage of times bit i changes (flips) between consecutive data frames within $P_{d,s/o}$. It is calculated as follows:

$$bfr_{n,s/o}[i] = \frac{1}{N-1} \sum_{j=1}^{N-1} \mathbb{I}(P_{n,s/o}[i][j] \neq P_{n,s/o}[i][j+1]) \quad (1)$$

where:

- N is the total number of frames with ID n in $M_{s/o}$,
- $P_{n,s/o}[i][j]$ is the value of the i -th bit in the payload of the j -th frame for CAN ID n under stationary or operational states,
- $\mathbb{I}(\cdot)$ is an indicator function that equals 1 if the argument is true (indicating a flip between consecutive frames) and 0 otherwise.

The bit-flipping rate vector $bfr_{n,s/o}$ summarizes the flipping probabilities for all bit indices within $P_{n,s/o}$. A $bfr_{n,s/o}[i]$ of 0% means the bits never changes within $P_{d,s/o}[i]$, while 100% means the bit flips between every pair of consecutive frames. This bit-flipping rate is essential for identifying signal boundaries and categorizing fields based on their roles in car-control actions.

1) *Signal Boundary Identification*: To systematically classify and represent the fields within $P_{d,s/o}$, a Signal Classification Vector $\tau_{d,s/o}$ is defined, where each element $\tau_{d,s/o}[i]$ represents the field type of the i -th bit in $P_{d,s/o}$, categorized into the following types:

- C : Constant fields – Bits that remain unchanged across all frames.
- κ_{inc} and κ_{dec} : Incrementing and decrementing counter fields – Bits that exhibit structured sequential behavior.

- F_i : Co-flipping fields – Bits that flip simultaneously with other bits.
- R : Randomly changing fields – Bits that do not fit any other classification.

The classification of each bit is based on its behavior across frames in $P_{d,s/o}$ determined by the following rules:

a) *Constant Fields (C)*: If all bits in $P_{d,s/o}[i]$ are identical, the i -th bit is classified as a constant bit, i.e. $\tau_{d,s/o}[i] = C$.

b) *Counter Fields (κ_{inc} and κ_{dec})*: Consecutive bits $P_{d,s/o}[i \sim j]$ form a counter field if they satisfy the following:

- The bit-flipping rate $bfr_{n,s/o}[i]$ of each higher-order bit k ($i \leq k < j$) is approximately half that of its adjacent lower-order bit $k+1$.
- During a higher-order bit flip: For incrementing counters (κ_{inc}): The lower-order bit flips from 1 to 0, indicating a carry. For decrementing counters (κ_{dec}): The lower-order bit flips from 0 to 1, indicating a borrow.

If these conditions hold, all bits in the range $[i, j]$ are classified as counter bits:

$$\tau_{d,s/o}[k] = \kappa_{inc} \text{ or } \kappa_{dec}, \quad \forall k \in [i, j] \quad (2)$$

c) *Co-flipping Fields (F_i)*: Bits in $P_{d,s/o}[i, P_{d,s/o}[j]$ and others form a co-flipping group if they:

- Have identical bit-flipping rates.
- Flip simultaneously across all frames in $P_{d,s/o}$.

Each group is assigned an identifier F_i , where i is the lowest index within the group:

$$\tau_{d,s/o}[k] = F_i, \quad \forall k \in \{i, j, \dots\} \quad (3)$$

d) *Randomly Changing Fields (R)*: Bits that do not fit any of the above categories are classified as randomly changing:

$$\tau_{d,s/o}[i] = R \quad (4)$$

The following pseudocode summarizes the classification process:

Algorithm 1 Pseudo-Code of the Classification Step	
1.	For each bit position i in range (0, 63):
2.	If $bfr_{d,s/o}[i] = 0$:
3.	$\tau_{d,s/o}[i] \leftarrow C$
4.	Else if $bfr_{n,s/o}[i] \approx 2 \times bfr_{n,s/o}[i+1]$:
5.	If CheckIncrementingCounter($P_{d,s/o}, i$):
6.	$\tau_{d,s/o}[i] \leftarrow \kappa_{inc}$
7.	Else if CheckDecrementingCounter($P_{d,s/o}, i$):
8.	$\tau_{d,s/o}[i] \leftarrow \kappa_{dec}$
9.	Else if CoFlippingDetected($P_{d,s/o}, i$):
10.	$\tau_{d,s/o}[i] \leftarrow F_k$
11.	Else :
12.	$\tau_{d,s/o}[i] \leftarrow R$
13.	Return $\tau_{d,s/o}$

For example, the classification result for CAN ID 273 in the stationary state ($\tau_{273,s}$) is: $\tau_{273,s} = [C, C, \kappa_{inc}, \kappa_{inc}, \kappa_{inc}, R, R, \dots]$. This illustrates how each bit in $P_{273,s}$ is assigned a type based on its observed behavior across frames.

2) *Related-ID Locating*: The signal classification vectors $\tau_{d,s}$ and $\tau_{d,o}$, representing the signal classifications for CAN ID d in stationary and operational states, are compared to determine whether d belongs to the related-ID set $\Sigma_{related}$. If there is any bit index i such that $\tau_{d,s}[i] \neq \tau_{d,o}[i]$, the ID is classified as related and included in $\Sigma_{related}$; otherwise, it is considered unrelated.

Figure 5 illustrates this process using $\tau_{2C2,s}$ and $\tau_{2C2,o}$ as an example. The vectors show differences in certain bit classifications, such as a transition from C to R, therefore $2C2 \in \Sigma_{related}$. In contrast, for $\tau_{273,s}$ and $\tau_{273,o}$, no differences are observed, so $273 \notin \Sigma_{related}$.

This direct comparison approach ensures that $\Sigma_{related}$ is constructed efficiently, capturing IDs most likely to be relevant for car-control actions.

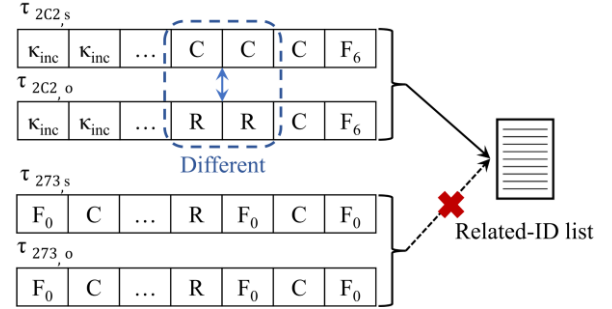


Fig. 5. Examples of Determining Related and Unrelated IDs Using Signal Classification Vectors

D. Fuzzed Data Generation and Injection

For each CAN ID d , fuzzed data generation begins by selecting an initial data frame from $P_{d,o}$, representing the payloads collected in the operational state. This initial frame becomes the first row of the fuzzed data matrix, denoted as $P_{d,fuzzed}[0]$. Each subsequent row $P_{d,fuzzed}[n]$ ($n \geq 1$) is derived iteratively from the previous row $P_{d,fuzzed}[n-1]$ by applying mutation rules based on $\tau_{d,o}$.

Given the previous generated frame $P_{d,fuzzed}[n-1]$, for each frame $P_{d,fuzzed}[n]$, which represents the new frame to be generated based on the previously generated frame $P_{d,fuzzed}[n-1]$, each bit i is mutated as follows:

If $\tau_{d,o}[i] = C$, the i -th bit remains constant:

$$P_{d,fuzzed}[n][i] = P_{d,fuzzed}[n-1][i]. \quad (5)$$

If $\tau_{d,o}[i] = R$, the i -th bit flips independently with a probability equal to $bfr_{d,o}[i]$:

$$P_{d,fuzzed}[n][i] = \begin{cases} 1 - P_{d,fuzzed}[n-1][i], & \text{with probability } bfr_{d,o}[i], \\ P_{d,fuzzed}[n-1][i], & \text{otherwise.} \end{cases} \quad (6)$$

If $\tau_{d,o}[i] = F_k$, all bits in the group F_k flip together, with a probability determined by $bfr_{d,o}[k]$, consistent across the group:

$$P_{d,fuzzed}[n][i] = \begin{cases} 1 - P_{d,fuzzed}[n-1][i], & \text{if group flip occurs,} \\ P_{d,fuzzed}[n-1][i], & \text{otherwise.} \end{cases} \quad (7)$$

If $\forall k \in [i, j]$, $\tau_{d,o}[k] = \kappa_{inc}$ or κ_{dec} , the bits spanning $i \sim j$ are mutated according to the following rules. For the least significant bit ($k = j$), the flipping probability is determined by its bit-flipping rate $bfr_{d,o}[j]$:

$$P_{d,fuzzed}[n][j] = \begin{cases} 1 - P_{d,fuzzed}[n-1][j], & \text{with probability } bfr_{d,o}[j], \\ P_{d,fuzzed}[n-1][j], & \text{otherwise.} \end{cases} \quad (8)$$

For higher-order bits ($i \leq k \leq j$), flipping is determined by whether a carry (for incrementing counters) or a borrow (for decrementing counters) propagates from lower-order bits:

$$P_{d,fuzzed}[n][k] = \begin{cases} 1 - P_{d,fuzzed}[n-1][k], & \text{if carry/borrow affects bit } k, \\ P_{d,fuzzed}[n-1][k], & \text{otherwise.} \end{cases} \quad (9)$$

Using these mutation rules, the fuzzed data matrix $P_{d,fuzzed}$ systematically explores variations in payloads for CAN ID d . After constructing $P_{d,fuzzed}$, the payloads are combined with frame headers and footers to create the complete fuzzed dataset $M_{d,fuzzed}$, which is injected into the vehicle's CAN bus.

If any $M_{d,fuzzed}$ triggers a specific car-control action ω_i , binary search is performed to isolate the exact frames responsible. The payloads of these frames are recorded as Φ_{d,ω_i} , an unordered set of payload vectors P , an unordered set of payload vectors for CAN ID d that independently trigger ω_i .

The set of all triggered responses ω_i is denoted as Ω_{tr} , a set encompassing all car-control actions observed during the injection process.

E. Control Bit Analysis

This step identifies the exact bits in the payload that are responsible for triggering a specific car-control action ω_i . The analysis has two key steps: (1) commonality analysis, which isolates potentially relevant bits, and (2) selective bit testing, which refines these relevant bits to identify the exact control bits. Together, these steps form a precise methodology for identifying and verifying control bits for ω_i .

In the commonality analysis, Φ_{d,ω_i} is analyzed. Here, each vector P in Φ_{d,ω_i} represents the payload of a single CAN frame that triggered ω_i . The analysis produces a marked vector χ_{d,ω_i} , which captures the consistent and variable bit positions across all payloads P . Each bit in χ_{d,ω_i} is defined as:

$$\chi_{d,\omega_i}[i] = \begin{cases} 1, & \text{if } P[i] = 1, \forall P \in \Phi_{d,\omega_i} \\ 0, & \text{if } P[i] = 0, \forall P \in \Phi_{d,\omega_i} \\ x, & \text{if } \bigoplus_{P \in \Phi_{d,\omega_i}} P[i] = 1. \end{cases} \quad (10)$$

Here, x indicates that the i -th bit varies across payloads, suggesting it is unrelated to ω_i . The marked vector χ_{d,ω_i} eliminates irrelevant bits (x) and retains only those (0s and 1s) potentially related to ω_i .

In the selective bit testing, the marked vector χ_{d,ω_i} is systematically tested to determine which bits directly influence ω_i . For each bit i marked 1 or 0 in χ_{d,ω_i} , a test vector $\bar{\chi}_{d,\omega_i}^{(i)}$ is generated by flipping the i -th bit in χ_{d,ω_i} , while all x positions are set to 0. In total, n test vectors $\bar{\chi}_{d,\omega_i}^{(i)}$ are produced, where n is the number of 1s or 0s in χ_{d,ω_i} . Each test vector $\bar{\chi}_{d,\omega_i}^{(i)}$ is injected as a CAN frame, and its effect is observed. If the injection of $\bar{\chi}_{d,\omega_i}^{(i)}$ fails to trigger ω_i , the i -th bit is identified as a control bit. This implies that flipping this bit disrupts ω_i .

The final control bit set C_{d,ω_i} , storing the indices of all identified control bits, is defined as:

$$C_{d,\omega_i} = \{i \mid \bar{\chi}_{d,\omega_i}^{(i)} \text{ fails to trigger } \omega_i\} \quad (11)$$

Through these steps, the control bit analysis provides precise identification of the control bits for ω_i , enabling bit-level resolution reverse engineering of car-control actions.

IV. PERFORMANCE EVALUATION

In this section, we evaluate the performance of our CAN bus reverse engineering method. Our experiments were

conducted on the Tesla Model 3 (2023) and the Leapmotor C11 and C10 (2024).

A. Experimental Setup

As described in section III. B, the data collection phase involves capturing CAN bus data ($M_{s/o}$) in two distinct states: a stationary state and an operational state. During the operational state, a series of car-control actions (Ω_{op} , defined in section III. B) were manually triggered. These car-control actions are the actions that we aim to reverse engineer and they form the foundation for subsequent analysis.

In our experiment on Tesla Model 3, Ω_{op} included the following car-control actions, as shown in Table II:

TABLE II. CAR-CONTROL ACTIONS TRIGGERED MANUALLY (Ω_{op}) DURING DATA COLLECTION PHASE

Window Up	Trunk Open	High Beam
Window Down	Trunk Close	Horn Activation
Multi-Level Wiper Control	Rearview Mirror Fold	Rearview Mirror Angle Adjustment
Left-turn Signal	Front Trunk Open	Door Lock
Right-turn Signal	Front Trunk Close	Door Unlock

B. Experimental Results

Through our methodology, we successfully obtained the control bits of a series of car-control actions, collectively defined as Ω_{tr} . The contents of Ω_{tr} are summarized in Table III. In our experiments, Ω_{op} consists of 8 manually triggered car-control actions, while Ω_{tr} includes a total of 43 actions, including both the predefined actions and additional function controls or abnormal responses discovered during the analysis.

TABLE III. CAR-CONTROL ACTIONS THAT CAN BE INTERPRETED

Vehicle Function Controls	Abnormal Responses
Window Up/ Down	Window Sensors Deception
Front/Back Trunk Open/ Close	Front Seat Sensors Deception
Keyless Driving	Display Reboot
Turn Signals	Headlight Flicker
Multi-Level Wiper Control	Battery Warning Light
Long/ Short Horn Activation	Air Conditioning Shutdown
Silent Mode for Media	Climate Control Malfunction
Glove Box Open/ Close	Internal Circulation Off
High/ Low Beam	Defrost Function Error
Window Gap Open/ Close	Light Sensors Deception
Door Lock/ Unlock	Volume Display Flickering
Brake Light	Start Status Detection
Double-flash	Screen Display Error
Volume Control	A Tiny Slit of Window Landed
	Air Conditioning Airflow Malfunction

As described in section III. B, the data collection phase involves capturing CAN bus data ($M_{s/o}$) in two distinct states: a stationary state and an operational state. During the operational state, a series of car-control actions (Ω_{op} , defined in section III. B) were manually triggered. These car-control actions are the actions that we aim to reverse engineer and they form the foundation for subsequent analysis.

The experimental results reveal several key insights:

1) *Coverage*: As shown in Table II and Table III, all 15 actions in Ω_{op} are fully contained within Ω_{tr} , confirming 100% coverage of the predefined car-control actions. This

demonstrates the effectiveness of our methodology in identifying control bits for these targeted actions.

2) *Additional Car-control Actions*: Beyond the 15 predefined actions, Ω_r includes an additional 14 function controls and abnormal responses, bringing the total number of car-control actions to 43. These unexpected entries arise from the fuzzed data generation process, which efficiently produces plausible yet novel data frames beyond the original dataset. Such frames may activate related but untested actions or trigger conflicts within the ECU, resulting in abnormal responses. These anomalies are particularly significant as they reflect vulnerabilities in the ECU's processing logic, including insufficient error handling or flawed prioritization mechanisms.

C. Comparison with Existing Methods

To establish the efficiency and effectiveness of our method, we compared it against several existing approaches as shown in Table IV below.

TABLE IV. COMPARISON WITH EXISTING METHODS

Existing work	Experimental vehicles	Functions Controls	Abnormal Responses	Resolution
Fowler et al. [19]	1	×	×	Moderate (frame-level)
Zhang et al. [20]	1	×	×	Not mentioned
Jia et al. [21]	1	LED indicating car lock	✓	Moderate (frame-level)
Fowler et al. [22]	×	Unlock the car door	✓	High (bit-level)
Zixiang Bi et al. [23]	1	×	×	Low(ID-level)
Zixiang Bi et al. [24]	1	20 functions including lighting, wipers, etc.	×	High (bit-level)
Our work	3	43 actions and some high-risk actions involved (shown in Table III)	✓	High (bit-level)

From Table IV, we can draw the following conclusions regarding our method's performance compared to others:

1) *Car-control Actions Identification*: Our method successfully reverse-engineered 43 car-control actions, achieving bit-level resolution for all identified actions. In contrast, most previous works, such as Fowler et al. [19, 22] and Jia et al. [21], primarily focused on frame-level or ID-level analysis, limiting their ability to identify precise control bits. Even Bi et al. [24], which achieved bit-level resolution, covered fewer actions than our method.

2) *Unexpected Responses*: Our approach extends beyond predefined car-control actions by efficiently uncovering additional related function controls and abnormal responses through fuzzed data generation. Previous works, such as Zhang et al. [20] and Bi et al. [24], did not explore unexpected responses, limiting their ability to detect unexpected ECU behaviors or potential conflicts.

3) *Applicability Across Vehicles*: Our method was validated on three real vehicle models, including the Tesla Model 3 and Leapmotor C11/C10, showcasing its robustness and generalizability. While some previous works, such as Fowler et al. [22] and Zhang et al. [20], were limited to a

single vehicle, our broader applicability demonstrates its potential for diverse automotive platforms.

These results highlight the strengths of our methodology in achieving higher resolution and more extensive coverage, compared to existing approaches.

D. Overhead of Our Approach

The overhead of our approach includes a total time of 2 minutes for data collection, 30 seconds for data preprocessing, 3 seconds for bit-flipping analysis, 15 seconds for fuzzed data generation, and 20 minutes for control bit analysis (manual effort involved).

Resource utilization remains low, with an average CPU usage of 6% and memory consumption of approximately 73 MB, demonstrating the efficiency of the method on standard hardware.

E. Wireless Automatic Reverse Engineering Tool Development

We developed an automatic reverse engineering tool based on a Raspberry Pi 4b, equipped with a 2-CH CAN FD HAT for CAN communication. This tool is capable of receiving commands wirelessly from a laptop, reverse engineering, and performing car-control actions remotely via the CAN bus shown in Figure 6.



Fig. 6. The Wireless Automatic Reverse Engineering Tool Connected to Tesla Model 3in

The compact design and wireless capability make this tool portable and suitable for on-site experiments, enabling efficient execution of reverse-engineering tasks.

V. CONCLUSION

This paper presents a systematic framework for reverse engineering CAN bus messages, achieving bit-level resolution. The proposed method has been validated across multiple vehicle models, including the Tesla Model 3 and Leapmotor C10 and C11, demonstrating versatility across different vehicle platforms. On a single vehicle Tesla, it successfully identified 43 car-control actions, showcasing its extensive coverage of car-control actions. Furthermore, the method's low resource consumption allows seamless integration into compact platforms like the Raspberry Pi, facilitating practical deployment in real-world automotive systems. It establishes a practical and systematic method for precise bit-level identification.

REFERENCES

- [1] Bozdal, Mehmet, et al. "Evaluation of can bus security challenges." *Sensors*, 2020, 2364
- [2] Lee, H., Choi, K., Chung, K., Kim, J., & Yim, K. Fuzzing can packets into automobiles. *IEEE Advanced Information Networking and Applications*, 2015, 817-821.
- [3] Greenberg, A. Hackers Remotely Kill a Jeep on the Highway—With Me in It. *Wired*, 2015. Available online: <https://www.wired.com/2015/07/hackers-remotely-kill-jeep-highway/>.
- [4] Weinmann, R.P.; Schmotzle, B. TBONE—A Zero-Click Exploit for Tesla MCUs. Kunnamon, 2021. Available online: <https://kunnamon.io/tbone/tbone-v1.0-redacted.pdf>.
- [5] Ezeobi, Uchenna, et al. "Reverse engineering controller area network messages using unsupervised machine learning." *IEEE Consumer Electronics Magazine*, 2020, 50-56.
- [6] Lee, H., Jeong, S. H., & Kim, H. K. OTIDS: A novel intrusion detection system for in-vehicle network by using remote frame. *Privacy, Security and Trust*, 2017, 57-5709.
- [7] Marchetti, M., & Stabili, D. READ: Reverse engineering of automotive data frames. *IEEE Transactions on Information Forensics and Security*, 2018, 1083-1097.
- [8] Pesé, M. D., Stacer, T., Campos, C. A., Newberry, E., Chen, D., & Shin, K. G. LibreCAN: Automated CAN message translator. *ACM Computer and Communications Security*, 2019, 2283-2300.
- [9] Choi, Wonsuk, et al. "An enhanced method for reverse engineering CAN data payload." *IEEE Transactions on Vehicular Technology*, 2021, 3371-3381.
- [10] Ji, Cheongmin, Taehyoung Ko, and Manpyo Hong. "CA-CRE: Classification Algorithm-Based Controller Area Network Payload Format Reverse-Engineering Method." *Electronics*, 2021, 2442.
- [11] Yang, K. ACRE: An Advanced CAN Signal Reverse Engineering Scheme. *Consumer Electronics and Computer Engineering*, 2023, 663-666.
- [12] Karamolegkos, Michail. Automotive protocol reverse engineering & car Internal Network Reconnaissance. MS thesis. 2021.
- [13] Kang, T. U., Song, H. M., Jeong, S., & Kim, H. K. Automated reverse engineering and attack for CAN using OBD-II. *IEEE Vehicular Technology Conference*, 2018, 1-7.
- [14] Blaauwendraad, Bram, and Vincent Kieberl. "Automated reverse-engineering of CAN messages using OBD-II and correlation coefficients." 2020, Available online: <https://rp.os3.nl/2019-2020/p103/report.pdf>
- [15] Song, Hyun Min, and Huy Kang Kim. "Discovering can specification using on-board diagnostics." *IEEE Design & Test*, 2020, 93-103.
- [16] Buscemi, A., Castignani, G., Engel, T., & Turcanu, I. A data-driven minimal approach for CAN bus reverse engineering. *IEEE Connected and Automated Vehicles Symposium*, 2020, 1-5.
- [17] Young, C., Svoboda, J., & Zambreno, J. Towards reverse engineering controller area network messages using machine learning. *IEEE World Forum on Internet of Things*, 2020, 1-6.
- [18] Fowler, D. S., Bryans, J., Cheah, M., Wooderson, P., & Shaikh, S. A. A method for constructing automotive cybersecurity tests, a CAN fuzz testing example. *IEEE Software Quality, Reliability and Security Companion*, 2019, 1-8.
- [19] Zhang, H., Huang, K., Wang, J., & Liu, Z. Can-ft: A fuzz testing method for automotive controller area network bus. *Computer Information Science and Artificial Intelligence*, 2021, 225-231.
- [20] Jia, Y., Yang, J., Wang, S., Wang, Y., Chen, H., & Ma, Z. Algorithm Research on Fuzzy Test Method of CAN Bus Based on Field Weight. *IEEE Power, Electronics and Computer Applications*, 2022, 311-318.
- [21] Fowler, D. S., Bryans, J., Shaikh, S. A., & Wooderson, P. Fuzz testing for automotive cyber-security. *IEEE/IFIP Dependable Systems and Networks Workshops*, 2018, 239-246.
- [22] Bi, Zixiang, et al. "Bit-level automotive controller area network message reverse framework based on linear regression." *Sensors*, 2022, 981.
- [23] Bi, Zixiang, et al. "A Method for Translating Automotive Body-Related CAN Messages Based on Labeled Bits." *Applied Sciences*, 2023, 1942.